

If your goal is to **build traffic in the UAE**, I'd avoid starting a generic "Dubai lifestyle" blog. It's very competitive. Instead, choose a specific audience + problem where you can publish lots of useful, searchable content.

Blog niches with strong potential in the UAE

- **Dubai/UAE cost-of-living**
 - "Cost of living in Dubai for a single person"
 - "Monthly expenses in Dubai in 2026"
 - "Cheapest areas to rent in Dubai"
 - "Dubai salary vs living expenses"
- **UAE jobs & careers**
 - "How to find a job in Dubai"
 - "CV format for UAE jobs"
 - "Highest-paying jobs in UAE"
 - "Interview questions for UAE companies"
 - "How to change jobs in the UAE"
- **Dubai restaurants & food**
 - "Best affordable restaurants in Dubai"
 - "Best Indian restaurants in Dubai"
 - "Best restaurants for families"
 - "Dubai restaurants under AED 100"
 - "Best breakfast places in Dubai"
- **UAE travel & weekend trips**
 - "Best places to visit in UAE"
 - "Dubai to Oman road trip"
 - "Best UAE beaches"
 - "Free places to visit in Dubai"
 - "2-day UAE itinerary"
- **UAE business & entrepreneurship**
 - "How to start a business in Dubai"
 - "Small business ideas in UAE"
 - "Free zones explained"
 - "E-commerce business in UAE"

- “How to get customers in Dubai”
- **Dubai deals & savings**
 - “Best supermarket deals this week”
 - “Affordable gyms in Dubai”
 - “Best mobile plans in UAE”
 - “Cheap activities in Dubai”
 - “Where to buy affordable furniture in Dubai”
- **UAE property**
 - “Best areas to rent in Dubai”
 - “Dubai rent prices by area”
 - “Buying vs renting in Dubai”
 - “Best areas for families”
 - “Dubai property guides for first-time buyers”

A particularly interesting strategy

If you're starting from zero, I'd consider a “**Practical UAE**” website rather than a broad lifestyle blog.

For example:

Everything someone moving to or living in the UAE needs to know — money, jobs, housing, food, transportation, services and weekend activities.

Then create clusters:

Moving to UAE

→ visas → housing → SIM cards → bank accounts → utilities → schools → transportation

Living in Dubai

→ rent → groceries → salaries → neighborhoods → gyms → restaurants → activities

Saving money

→ supermarket comparisons → phone plans → bank accounts → discounts → affordable restaurants

Jobs

→ CVs → interviews → salaries → industries → job-search resources

This gives you hundreds of potential articles while keeping the website focused.

The key to getting traffic

Don't build articles around topics that are merely interesting. Build them around **questions people actually search for**.

Instead of:

“Why Dubai Is Amazing”

write:

“How Much Does It Cost to Live in Dubai in 2026?”

Instead of:

“Dubai's Best Restaurants”

write:

“15 Restaurants in Dubai Where You Can Eat for Under AED 50”

The second type has much clearer search intent.

If you tell me **your budget, your skills (writing/SEO/AI/etc.), and whether you want UAE-only or global traffic**, I can give you **3 blog niches with a 6-month content plan and 50 article ideas for each**.

Budget: 500-1000 AED

Skills: Senior Software Engineer/AI
Global

Given your **AED 500–1,000 budget, Senior Software Engineer + AI skills**, and goal of **global traffic**, I would change the recommendation significantly.

I would **not** build a general UAE blog. Your strongest asset is your technical experience. You can build a global site around **practical AI/software engineering**, with content that demonstrates real implementations rather than generic AI news.

The current developer landscape supports this direction: Stack Overflow's 2025 survey found that 84% of developers use or plan to use AI tools, while 46% said they distrust AI output accuracy. That creates an opportunity for content focused on **testing, debugging, architecture, production implementation, and verification**, rather than simply generating AI-written tutorials.  Stack Overflow... +1

My suggested niche

Practical AI Engineering for .NET Developers

Think of the website as:

Build real AI-powered applications with .NET, SQL Server, RAG, vector databases, LLMs and AI agents.

This fits your existing skills extremely well.

Your technology stack gives you a particularly useful combination:

.NET 8 + C# + SQL Server + React + Ollama + RAG + Qdrant + AI Agents + Docker

That's much more differentiated than another "AI tools and news" website.

1. Your 5 main content pillars

Pillar 1 — AI + .NET

Examples:

- Build an AI chatbot with ASP.NET Core
- Connect Ollama to .NET 8
- Build an LLM API with C#
- Build RAG with ASP.NET Core
- .NET + Qdrant semantic search
- Build AI agents with .NET
- Build streaming AI responses with SignalR
- Build an AI document-processing API

This should be your foundation.

Pillar 2 — SQL Server + AI

This could become a particularly strong sub-niche.

Examples:

- Can an LLM query SQL Server safely?
- Build a Text-to-SQL agent with .NET
- SQL Server + RAG
- SQL Server vs vector database for AI
- How to give an LLM database schema information
- Build a natural-language database assistant
- Secure Text-to-SQL architecture
- Prevent destructive SQL generated by an LLM

- Add RBAC to an AI database agent
- SQL Server + Qdrant architecture

This is much more valuable than generic "What is ChatGPT?" articles.

2. RAG should be one of your biggest categories

You could create an entire learning hub:

RAG

- What is RAG?
- Embeddings explained
- Vector databases
- Chunking
- Metadata filtering
- Similarity search
- Hybrid search
- Reranking
- Qdrant
- RAG with .NET
- RAG with SQL Server
- RAG evaluation
- RAG security
- Production RAG architecture

Then build progressively harder projects.

For example:

```
Beginner
  ↓
Simple RAG
  ↓
PDF RAG
  ↓
SQL + RAG
  ↓
Hybrid RAG
  ↓
Agentic RAG
  ↓
Production RAG
```

That gives you a **content moat** instead of publishing random articles.

3. AI Agents

This is another strong pillar.

Topics:

- What is an AI agent?
- AI agent vs chatbot
- Tool calling
- Agent memory
- Planning
- Multi-agent architecture
- Agent orchestration
- Agent security
- Agent observability
- Agent evaluation
- AI agents with .NET
- AI agents with SQL Server
- AI agents with Ollama
- AI agents with LangChain
- AI agents with Semantic Kernel
- CrewAI vs LangGraph vs Semantic Kernel

The Stack Overflow survey shows that AI agents are still developing rather than being universally adopted: 52% of respondents either don't use agents or use simpler AI tools, and 38% had no plans to adopt agents. That makes **practical educational material** particularly useful rather than assuming everyone already understands agents.

 Stack Overflow...

4. AI-assisted software development

This is another area I'd seriously pursue.

You could cover:

Claude

- Claude Code tutorials
- Claude + .NET
- Claude + React

- Claude for API development
- Claude for refactoring
- Claude for testing
- Claude for code review

GitHub Copilot

- Copilot for .NET
- Copilot for SQL
- Copilot for React
- Copilot agent mode
- Copilot for unit testing

Other coding agents

- Cursor
- Gemini
- OpenAI coding tools
- local LLM coding
- Ollama

The demand is there: Stack Overflow reports that 84% of respondents use or plan to use AI tools in development, while 66% identified "almost right" AI solutions as their biggest frustration.  Stack Overflow...

That gives you an excellent content angle:

Don't just show developers how to use AI. Show them how to verify and fix what AI produces.

5. Real production case studies

This is where I think you can differentiate yourself the most.

Instead of writing:

"What is RAG?"

build:

Project: AI Real Estate Assistant

React



ASP.NET Core API



AI Orchestrator



SQL Server	Qdrant
structured	embeddings
data	documents



Ollama



Response

Then create 20–30 articles from that one project.

For example:

1. Architecture
2. Database design
3. Embeddings
4. Document ingestion
5. Qdrant setup
6. Chunking
7. Retrieval
8. SQL queries
9. Prompt engineering
10. Tool calling
11. Agent design
12. Security
13. Logging
14. Evaluation
15. Docker deployment
16. React integration
17. Production optimization

One project becomes an entire **content ecosystem**.

What I would NOT do

With your budget, I wouldn't start with:

- ✗ General technology news
- ✗ General AI news

- ❌ Cryptocurrency
- ❌ Generic "make money online"
- ❌ General UAE lifestyle
- ❌ Generic programming tutorials
- ❌ "Top 10 AI tools" articles
- ❌ AI-generated news aggregation

Those areas require either significant authority, huge publishing volume, or strong distribution.

Your website positioning

I'd make the positioning very specific.

Something like:

Practical AI Engineering

Build production-ready AI applications with .NET, C#, SQL Server, RAG, vector databases and AI agents.

Your target audience:

```
.NET Developers
    ↓
Backend Developers
    ↓
Software Engineers
    ↓
AI Engineers
    ↓
Technical Leads
    ↓
Architects
```

And your global audience isn't restricted to the UAE.

Budget: AED 500–1,000

You actually don't need much.

I'd allocate roughly:

Item	Budget
Domain	AED 40–100/year
Hosting	AED 150–300/year
WordPress/website setup	AED 0–200
Logo/design	AED 0–100
SEO tools	AED 0 initially
Email/newsletter	AED 0 initially
Content	Your own expertise
Total	~AED 200–700

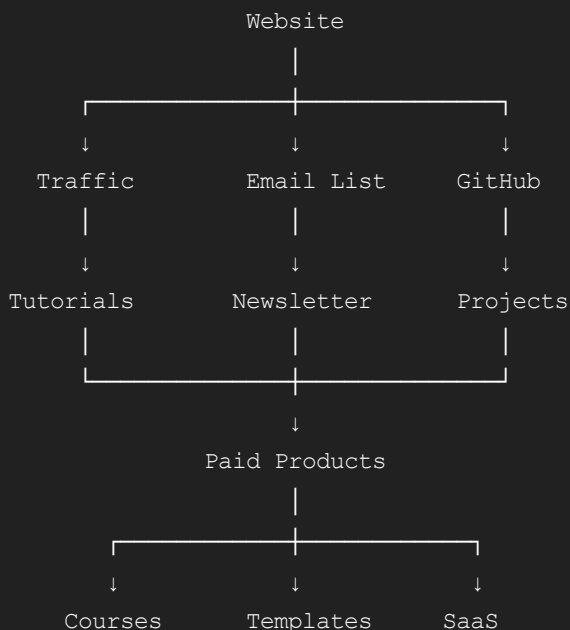
Keep the remaining budget for experiments rather than spending everything immediately.

You already have a major advantage: **you can build the website and technical demos yourself.**

The business model

Don't depend only on AdSense.

I'd build toward:



Potential products later:

- .NET AI starter kit
- RAG starter project
- AI agent boilerplate
- SQL-to-AI agent
- Production RAG template
- Prompt/testing framework
- AI architecture templates
- paid course
- consulting
- SaaS tools

This is where the website can eventually become a **business**, rather than simply an advertising site.

Your first 6 months

I'd structure the site like this:

Month 1 — Foundation

Create:

- Domain
- Website
- About
- GitHub
- Newsletter
- AI/.NET category
- RAG category
- AI Agents category

Publish **10 high-quality foundational articles**.

Month 2 — .NET + AI

Publish approximately:

12–15 articles

Examples:

ASP.NET Core + Ollama
ASP.NET Core + OpenAI
ASP.NET Core + Claude
.NET embeddings
.NET vector search
.NET RAG
.NET AI chatbot
.NET AI agents

Month 3 — RAG project

Build one complete project:

Production RAG Application with .NET + SQL Server + Qdrant + Ollama

Then turn the project into **15–20 articles**.

Month 4 — AI Agents

Build:

AI Database Agent

Example:

```
User:
"Show me all pending orders from Dubai."

↓

AI Agent

↓

Schema discovery

↓

SQL generation

↓

SQL validation

↓
```

SQL Server

↓

Result

↓

LLM explanation

This is particularly aligned with your existing technical experience.

Month 5 — Developer AI tools

Create practical comparisons/tutorials around:

- Claude
- ChatGPT
- GitHub Copilot
- Cursor
- Ollama
- local LLMs

But don't just review them.

Build something with each one.

Month 6 — Monetization

Start turning your best material into:

- downloadable projects
 - templates
 - ebooks
 - mini-courses
 - GitHub premium repositories
 - consulting
 - SaaS experiments
-

The important part: don't become an AI content farm

This is probably the biggest warning I'd give you.

Don't do:

ChatGPT

↓

Generate 1,000 articles

↓

Publish

↓

Wait for Google

Instead:

Your engineering experience

↓

Real problem

↓

Build solution

↓

Test it

↓

Measure it

↓

Document it

↓

Publish

↓

GitHub project

↓

Newsletter

↓

Product

Your engineering experience becomes the differentiator.

And there's a particularly good reason for that right now: developers are increasingly using AI, but trust remains a major issue. Stack Overflow's survey found 46% distrust AI output accuracy, while only 33% trust it.  Stack Overflow...

So your positioning could effectively be:

AI tutorials tested by a real software engineer.

That's much more compelling than another website producing generic AI articles.

If I were starting this with your exact profile

I'd choose:

Niche:

AI Engineering for .NET Developers

Core technologies:

.NET 8 + C# + SQL Server + React + Ollama + Qdrant + RAG + AI Agents

Audience:

Global software developers

Business model:

SEO → GitHub → newsletter → templates/courses → SaaS/consulting

Initial investment:

AED 300–700

First major project:

Natural Language SQL + RAG + AI Agent using .NET + SQL Server + Qdrant + Ollama

That project can become the foundation for **dozens of articles**, **GitHub repositories**, **videos**, **tutorials** and eventually a **paid product**.

If you want, the next step should be to design the **actual website/business from zero**:

domain/name ideas → site structure → 100 SEO article topics → first 20 articles → monetization plan → 6-month execution roadmap.

Yes

Yes. Based on your **AED 500–1,000 budget + senior .NET/AI background + global audience**, I'd build this as a **technical authority site**, not a conventional blog.

The opportunity is reasonable: Stack Overflow's 2025 survey covered 49,000+ developers across 177 countries, and 84% said they use or plan to use AI tools in development. At the same time, 46% distrust AI output accuracy and 66% report frustration with AI solutions that are "almost right." That strongly supports a content angle around **real implementation + verification + debugging**, rather than generic AI-generated tutorials.  Stack Overflow... +1

1. The business concept

Working positioning

Practical AI Engineering

Build production-ready AI applications with .NET, C#, SQL Server, RAG, vector databases and AI agents.

Your audience:

```
.NET Developers
  ↓
Backend Developers
  ↓
Software Engineers
  ↓
AI Engineers
  ↓
Technical Leads / Architects
```

The site should answer:

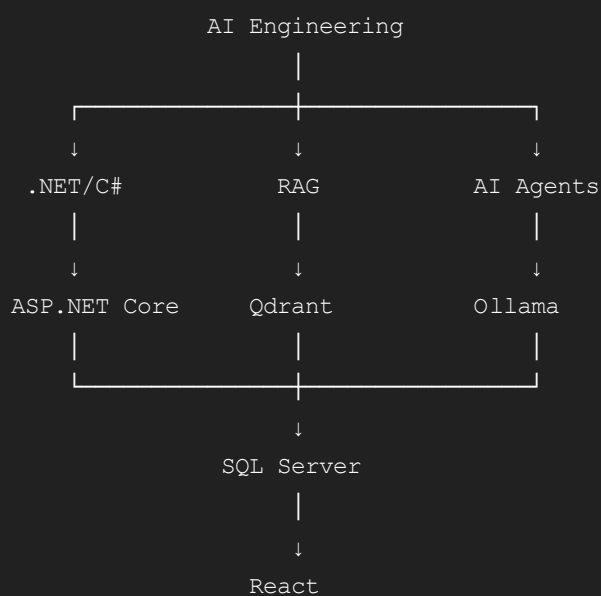
“How do I actually build this?”

rather than:

“What is AI?”

2. Your technology focus

I would make your stack the center of the site:



This is a much stronger identity than simply calling yourself an "AI blog."

3. Website structure

I'd keep the first version very simple.

```
Home
|
├── AI + .NET
|
├── RAG
|
├── AI Agents
|
├── SQL + AI
|
├── AI Developer Tools
|
├── Projects
|
├── Tutorials
|
├── Resources
|
└── About
```

AI + .NET

```
ASP.NET Core + AI
C#
LLM APIs
Ollama
Semantic Kernel
AI APIs
Streaming
SignalR + AI
```

RAG

```
RAG Fundamentals
Embeddings
Chunking
Vector Databases
Qdrant
Hybrid Search
Reranking
RAG Evaluation
Production RAG
```

AI Agents

Agent Fundamentals
Tool Calling
Memory
Planning
Multi-Agent Systems
Agent Security
Agent Evaluation
Agent Observability

SQL + AI

This could become your signature category:

Text-to-SQL
SQL Server + LLM
Database Agents
Schema Discovery
SQL Validation
Natural Language Queries
SQL + RAG
AI Database Security

4. Your first flagship project

Don't start by writing 100 unrelated articles.

Build **one serious project**.

I'd call it something like:

AI Database Agent

User asks:

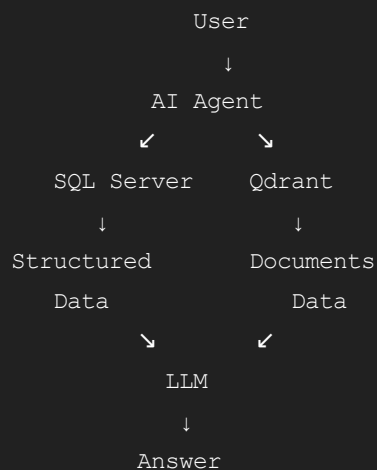
"Show me all pending orders from Dubai created this month."

System:

React
↓
ASP.NET Core API
↓
AI Agent
↓
Schema Discovery
↓
SQL Generation

↓
SQL Validation
↓
SQL Server
↓
Result
↓
LLM Explanation
↓
React

Then add RAG:



This project can become the **center of your entire website**.

5. Turn one project into 30+ articles

For example:

Architecture

1. How to Build an AI Database Agent
2. AI Database Agent Architecture
3. ASP.NET Core AI Agent Architecture
4. SQL Server + LLM Architecture

Database

5. How LLMs Understand Database Schemas
6. Give an AI Agent Access to SQL Server
7. SQL Server Schema Discovery with C#
8. Database Metadata for AI Agents

Text-to-SQL

9. What Is Text-to-SQL?
10. Build Text-to-SQL with .NET
11. Validate AI-Generated SQL
12. Prevent DELETE/UPDATE From AI Agents
13. SQL Injection Risks in Text-to-SQL
14. Restrict AI Agents to SELECT Queries

RAG

15. Add RAG to an AI Database Agent
16. SQL Server + Qdrant
17. Embeddings for Database Documentation
18. Semantic Search with Qdrant
19. Metadata Filtering
20. Hybrid SQL + Vector Search

Agent

21. Add Tool Calling
22. Add Agent Memory
23. Add Multiple Tools
24. Add Query Planning
25. Add Agent Logging
26. Add Agent Evaluation

Production

27. Secure an AI Database Agent
28. AI Agent RBAC
29. AI Agent Monitoring
30. AI Agent Error Handling
31. AI Agent Cost Optimization
32. Deploy AI Agent with Docker

Now you aren't trying to constantly invent new topics.

6. Your first 20 articles

I'd start with these.

#	Article
1	How to Build an AI Application with ASP.NET Core

#	Article
2	How to Run Ollama Locally with .NET
3	Build a Chat API with ASP.NET Core and an LLM
4	What Is RAG? A Practical Developer Guide
5	Build Your First RAG Application with .NET
6	What Are Embeddings? Explained for .NET Developers
7	Qdrant Vector Database Explained
8	Build Semantic Search with .NET and Qdrant
9	SQL Server vs Vector Database for AI Applications
10	How LLMs Work With SQL Server
11	Build a Text-to-SQL Application with ASP.NET Core
12	Build a Secure SQL Agent
13	How to Validate AI-Generated SQL
14	Build an AI Database Agent with .NET
15	Add RAG to an AI Database Agent
16	Build an AI Agent With Tool Calling
17	AI Agent Memory Explained
18	Build an AI Agent With Ollama
19	AI Agent Security: What Developers Need to Know
20	Production Architecture for .NET AI Applications

7. Your content formula

Every technical article should follow roughly this structure:

```
Problem
↓
Why it matters
↓
Architecture
↓
Prerequisites
↓
Implementation
↓
Code
↓
Test
↓
Common errors
↓
Security considerations
↓
Performance
↓
Production considerations
↓
GitHub repository
```

This is important because **your differentiator is not merely explaining concepts.**

It's:

I built it. Here's the architecture. Here's the code. Here's what broke. Here's how I fixed it.

That addresses the exact frustration developers report with AI-generated code.

 Stack Overflow...

8. GitHub becomes part of the strategy

Create a GitHub organization or account dedicated to your projects.

For example:

```
github.com/yourbrand

├─ ai-dotnet-starter
├─ dotnet-rag
├─ qdrant-dotnet
├─ ai-sql-agent
├─ text-to-sql-dotnet
```

```
└─ ollama-dotnet
└─ ai-agent-dotnet
└─ production-rag
```

GitHub's free plan currently includes unlimited public/private repositories, so you don't need to spend your initial budget there.  GitHub

Every article should have:

```
Article
  ↓
GitHub Repository
  ↓
Working Demo
  ↓
Newsletter signup
```

That's much stronger than a standalone blog.

9. YouTube — but don't start with a separate strategy

You don't need to become a YouTuber immediately.

For each major project:

```
Build project
  ↓
Write article
  ↓
Record 5-10 minute demo
  ↓
YouTube
  ↓
Embed video in article
```

One piece of engineering work produces:

1 project + 5–15 articles + 1–3 videos + GitHub repository + social posts.

10. Newsletter

Create something simple.

Example:

"AI Engineering Weekly"

Once per week:

AI Engineering Weekly #001

- .NET AI tutorial
- RAG technique
- AI tool worth testing
- GitHub project
- One production lesson

Don't worry about monetizing the newsletter initially.

Your goal is:

SEO visitor → **email subscriber**

because search traffic isn't something you control.

11. Monetization roadmap

Don't put ads everywhere from day one.

I'd use this progression:

Stage 1

Free articles

+

GitHub

+

Newsletter

Stage 2

Free articles

+

GitHub

+

Newsletter

+

Premium templates

Stage 3

Templates
+
Courses
+
Starter kits

Stage 4

SaaS
+
Consulting
+
Enterprise products

Potential products:

Product 1

Production RAG Starter Kit

ASP.NET Core
+
React
+
Qdrant
+
SQL Server
+
Ollama/OpenAI
+
Authentication
+
Logging
+
Docker

Product 2

AI SQL Agent Starter Kit

Natural Language
↓
LLM
↓
Schema
↓
SQL
↓

Validation



SQL Server

Product 3

.NET AI Agent Boilerplate

That could eventually become much more valuable than advertising.

12. Domain/name strategy

I wouldn't lock the brand to:

- .NET
- C#
- Ollama
- OpenAI
- RAG

Those technologies can change.

Instead, create a broader engineering brand.

Some **working-name** directions:

Option A — AI-focused

PracticalAI Engineering

Option B — developer-focused

AI Dev Lab

Option C — engineering-focused

BuildAI Systems

Option D — architecture-focused

AI Architecture Lab

Option E — neutral

Engineering Intelligence

These are **concept directions**, not claims that the domains/trademarks are available. Before registering anything, check the domain and trademark situation.

I'd personally make the brand broader than .NET so that you can eventually write:

```
.NET
Python
TypeScript
React
AI
RAG
Agents
MCP
LLMs
Cloud
```

without rebranding.

13. Your SEO strategy

Don't target only huge keywords like:

"AI"

or:

"RAG"

You'll get crushed by established sites.

Instead target **long-tail developer problems**.

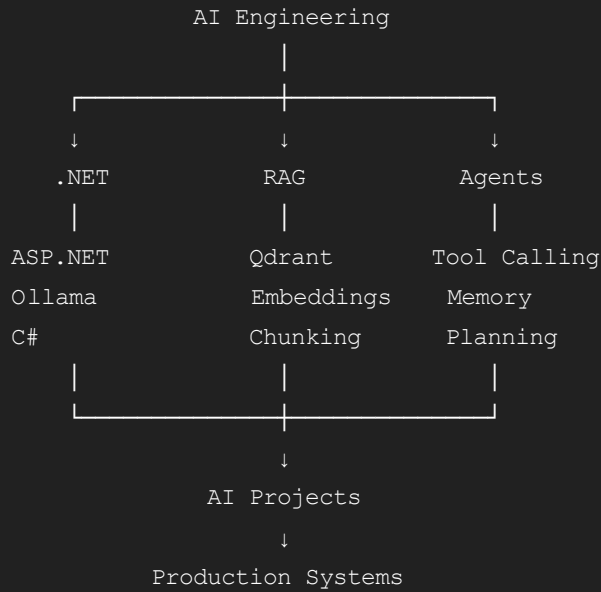
For example:

```
"qdrant with asp.net core"
"ollama c# example"
"sql server text to sql"
"rag asp.net core"
"qdrant .net"
"llm sql server"
"ai database agent"
"asp.net core ai chatbot"
"ollama asp.net core"
"semantic search c#"
```

Then gradually build authority toward broader terms.

14. Your article hierarchy

Think:



This creates **topical authority** instead of random SEO pages.

15. Your first 90 days

Days 1–7

Build:

Domain
Website
GitHub
Analytics
Search Console
Newsletter

Create the basic categories.

Don't spend weeks designing the logo.

Days 8–30

Build the first version of:

AI Database Agent

Technology:

React
ASP.NET Core
SQL Server
Ollama
Qdrant
Docker

Document everything.

Days 31–60

Publish:

15–20 technical articles

while improving the project.

Also create:

- GitHub repo
 - architecture diagrams
 - screenshots
 - demo video
-

Days 61–90

Publish another:

15–20 articles

Then launch:

Free resource

50 Practical AI Architecture Patterns for .NET Developers

Use it to collect email subscribers.

16. Budget

I'd actually spend **less than your maximum initially**.

First investment

Domain	~AED 50-100
Hosting	~AED 150-300
Email	~AED 0
GitHub	~AED 0
Analytics	~AED 0
SEO	~AED 0
Design	~AED 0
Initial	~AED 200-400

Keep the rest available.

Don't spend AED 1,000 just because you have AED 1,000.

Your biggest investment should be **engineering time**, not software subscriptions.

17. The most important differentiation

There are already thousands of articles saying:

"What is RAG?"

Your site should instead say:

"I built RAG with ASP.NET Core + Qdrant + SQL Server. Here's the architecture, implementation, failure cases, performance considerations and complete GitHub project."

And when AI generates something incorrectly, **show the correction**.

That is particularly valuable because the 2025 developer survey found that AI adoption is high but trust is considerably lower; 75% said they would still ask another person for help when they don't trust an AI answer.  Stack Overflow... +1

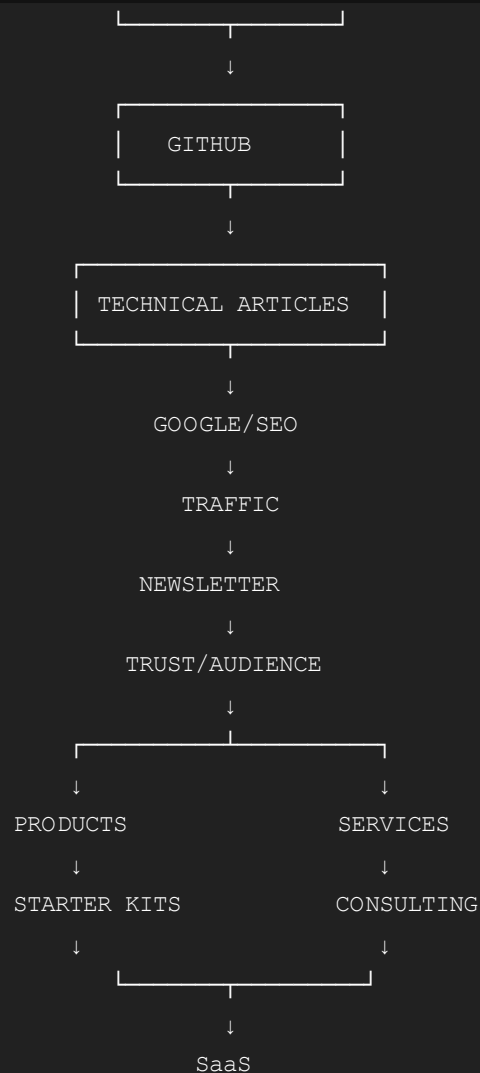
That can become the philosophy of the entire site:

Build. Test. Verify. Explain.

18. Your eventual flywheel

The whole business becomes:

REAL PROJECT



This is the model I'd build around your profile.

And there's another useful signal: more than 36% of Stack Overflow respondents reported learning AI programming or AI-enabled tooling in the prior year, while technical documentation was the most-used learning resource.  Stack Overflow...

So rather than trying to compete with news sites, build a **developer learning + implementation platform**.

Recommended next step

I would now move from strategy to execution and define **Phase 1: the actual website** — including **10–15 brand/domain candidates**, the exact homepage structure, navigation, category structure, About page positioning, GitHub organization structure, and the first flagship AI Database Agent project architecture.

For building AI-engineering blog traffic, here's an SEO platform option.



SEO for Google & AI
Grow organic traffic from Gemini & ChatGPT

yes

Great. Let's turn the idea into an **actual launch plan**.

One adjustment from my previous answer: I would make **“Practical AI Engineering” the editorial positioning, not necessarily the brand/domain name**. The brand should be broad enough to survive changes in AI tooling.

The market signal is good for this direction: Stack Overflow's 2025 survey says 84% of developers use or plan to use AI tools, while 46% distrust their accuracy; 66% cite “almost right” AI solutions as a major frustration.  Stack Overflow... Google also explicitly recommends people-first content demonstrating original information and first-hand expertise rather than mass-produced search-engine content.  Google for Dev...

Phase 1 — Build the foundation

1. Brand direction

I would shortlist these styles:

Name direction	Positioning
AI Dev Lab	Practical experiments and projects
Build AI Systems	Production AI engineering
Practical AI Engineering	Tutorials + architecture
AI Engineering Lab	Experimental/technical
BuildAI Systems	Products + engineering
Code & Intelligence	Broader long-term brand
Engineering AI	Professional/architect-oriented
AI Stack Lab	AI infrastructure + development

My preferred direction

AI Dev Lab

Tagline:

Build. Test. Verify. Ship AI.

Why I like this direction:

- Doesn't lock you to .NET.
- Doesn't lock you to a particular LLM.
- Works for articles.
- Works for GitHub.
- Works for YouTube.
- Works for future products/SaaS.
- Can eventually cover Python, TypeScript, .NET, agents, RAG, MCP, etc.

Before buying anything, check **domain availability and trademarks**. Don't spend money on a name until you've verified both.

2. Homepage

Keep the homepage extremely focused.

Hero

Build AI Systems That Actually Work

Practical tutorials, architectures and production projects for developers building AI applications.

[Explore Tutorials]

[View Projects]

Then:

Featured project

FEATURED PROJECT

AI Database Agent

Ask questions about your database using natural language.

ASP.NET Core

SQL Server

Ollama

Qdrant

React

[\[View Architecture\]](#)

[\[View GitHub\]](#)

Then:

What you'll learn

AI + .NET

Build AI applications with C# and ASP.NET Core.

RAG

Build semantic search and retrieval systems.

AI Agents

Build agents that can use tools and data.

SQL + AI

Connect LLMs to real enterprise databases.

Production AI

Security, evaluation, observability and deployment.

Then:

Latest tutorials

Show the latest 6 articles.

Then:

Newsletter

AI Engineering Weekly

One practical AI technique, one working project and one engineering lesson every week.

3. Navigation

Keep it to:

Home

AI + .NET

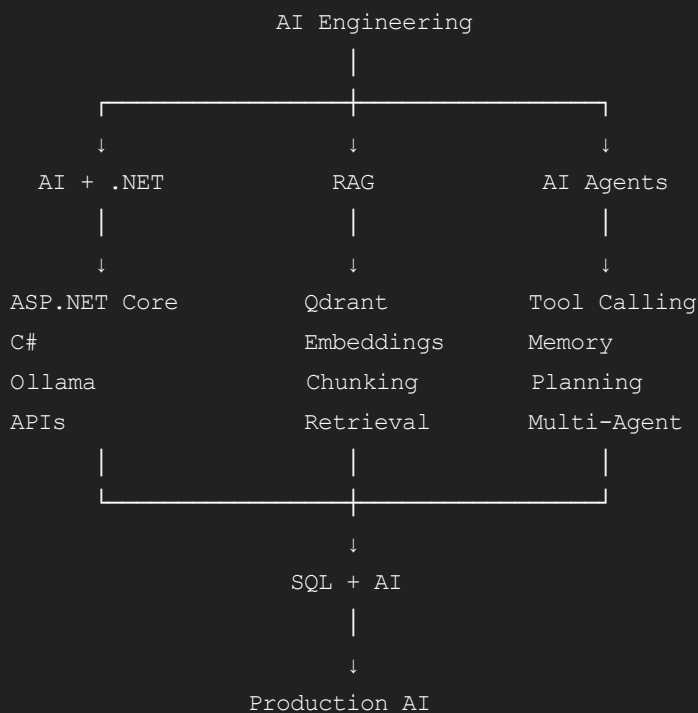
RAG

AI Agents

Don't create 25 categories at launch.

4. Your content architecture

This is important for SEO and usability.



This gives you a coherent topical structure instead of a collection of unrelated articles.

5. Your first flagship project

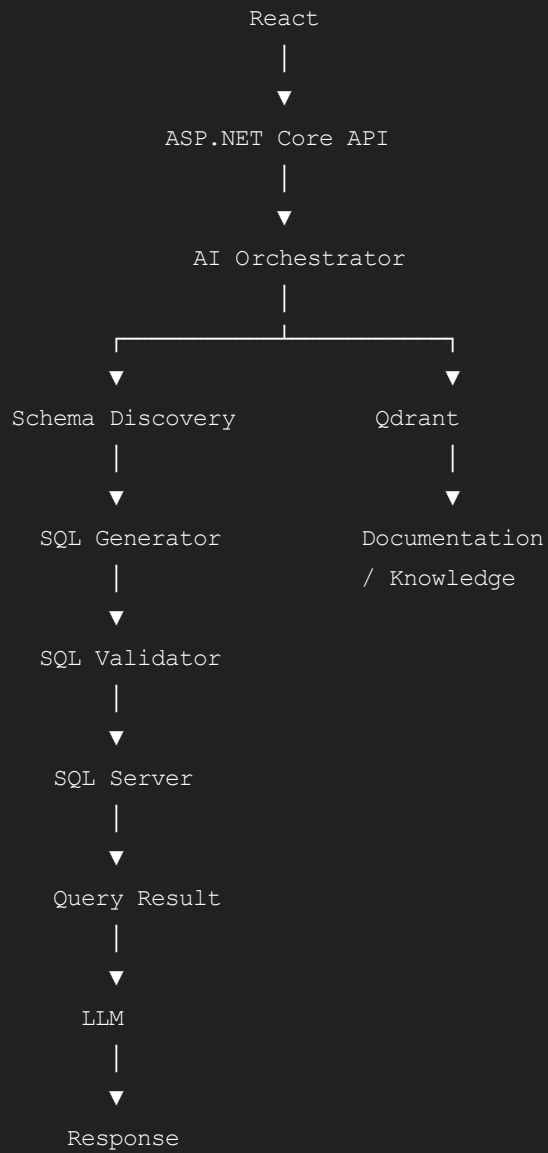
This should be your **anchor project**.

AI Database Agent

User

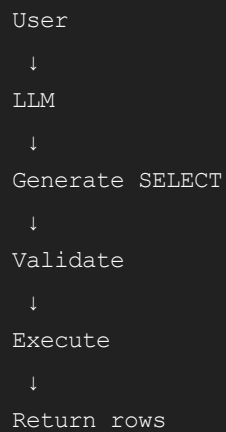
"Show me all pending orders from Dubai
created in the last 30 days."

Architecture



Your first version should **not** be a fully autonomous agent.

Start safely:



Then evolve it.

6. Version roadmap

V1 — Simple Text-to-SQL

```
graph TD
    Question --> LLM
    LLM --> SQL
    SQL --> Validation
    Validation --> SQL_Server[SQL Server]
```

V2 — Schema-aware

```
graph TD
    Question --> Schema_discovery[Schema discovery]
    Schema_discovery --> Relevant_tables[Relevant tables]
    Relevant_tables --> LLM
    LLM --> SQL
```

V3 — Secure

Add:

- SELECT-only
- table whitelist
- column restrictions
- query timeout
- row limits
- parameter validation
- audit logging

V4 — RAG

```
graph TD
    Question --> Split1[ ]
    Split1 --> SQL
    Split1 --> Qdrant
    SQL --> Join1[ ]
    Qdrant --> Join1
    Join1 --> Output[ ]
```

↓
LLM

V5 — Agent

Add tools:

```
SQL Tool
Search Tool
Documentation Tool
Calculation Tool
```

Now you have a real AI engineering project.

7. GitHub organization

Create:

```
AI-Dev-Lab/
```

Repositories:

```
ai-database-agent
dotnet-ai-starter
dotnet-rag
qdrant-dotnet
ollama-dotnet
text-to-sql-dotnet
ai-agent-dotnet
production-rag
```

You don't need a paid GitHub plan for this initial strategy. GitHub Free currently provides unlimited public and private repositories, and public repositories can use GitHub Actions within the included allowance.  GitHub +1

8. Repository structure

For `ai-database-agent`:

```
ai-database-agent/
|
├─ src/
|   └─ Api/
|   └─ Application/
```

```
|   ├── Domain/
|   ├── Infrastructure/
|   └── AI/
|
|   ├── tests/
|   |
|   ├── docs/
|   |   ├── architecture.md
|   |   ├── security.md
|   |   └── database.md
|   |
|   ├── docker/
|   |
|   ├── README.md
|   ├── docker-compose.yml
|   └── LICENSE
```

This also lets you demonstrate **Clean Architecture + AI**, which is much more valuable than a simple chatbot demo.

9. First 10 articles

I would actually publish these in this order.

Article 1

How to Build AI Applications with ASP.NET Core

Purpose:

Establish your core technology stack.

Article 2

Run Ollama Locally and Connect It to ASP.NET Core

Show:

```
ASP.NET Core → Ollama → LLM
```

Article 3

Build Your First AI Chat API with .NET

Simple implementation.

Article 4

What Is RAG? A Practical Guide for Software Developers

Don't make it academic.

Use:

```
Documents
  ↓
Chunks
  ↓
Embeddings
  ↓
Qdrant
  ↓
Similarity Search
  ↓
LLM
```

Article 5

Build a RAG Application with ASP.NET Core and Qdrant

Now readers have a real implementation.

Article 6

How LLMs Work With SQL Server

Explain:

```
LLM ≠ Database

LLM
  ↓
SQL
  ↓
Database
```

Article 7

Build Text-to-SQL with ASP.NET Core

Now introduce the flagship project.

Article 8

How to Safely Execute AI-Generated SQL

This is one of the articles I'd put significant effort into.

Cover:

- SELECT-only
 - validation
 - allowlists
 - row limits
 - timeouts
 - permissions
 - audit logs
-

Article 9

Build an AI Database Agent With .NET and SQL Server

This becomes your flagship tutorial.

Article 10

Add RAG to an AI Database Agent

Now combine:

```
.NET
+
SQL Server
+
Qdrant
+
Ollama
+
RAG
```

This is the point where your website starts developing a recognizable technical identity.

10. Your article template

Every serious tutorial should have:

```
# Title

Short introduction

## What We Are Building

Architecture diagram

## Technologies

## Prerequisites

## Step 1 – Setup

## Step 2 – Architecture

## Step 3 – Implementation

## Step 4 – Testing

## Step 5 – Common Errors

## Step 6 – Security

## Step 7 – Performance

## Step 8 – Production Considerations

## Complete Source Code

GitHub link

## What We Learned

## What's Next
```

The “**Common Errors**” section is particularly important.

For example:

`LIMIT 100` doesn't work in SQL Server.

Then explain:

SQL

```
SELECT TOP 100 ...
```

instead.

Those real implementation details are exactly what make the article useful.

11. Don't hide failures

This can become your brand.

Instead of pretending everything worked perfectly:

```
Attempt 1
❌ Generated MySQL LIMIT syntax

Attempt 2
❌ Wrong table relationship

Attempt 3
❌ Missing status lookup

Attempt 4
✅ Schema-aware SQL generation
```

That's genuinely useful to developers.

It also aligns with the current developer experience: Stack Overflow reports that 66% of developers' biggest AI frustration is output that is almost right but not quite, and 45% report that debugging AI-generated code can be more time-consuming.  Stack Overflow...

Your content can therefore be built around:

Here's what AI generated. Here's what was wrong. Here's how we made it production-safe.

12. SEO strategy

Don't try to rank immediately for:

AI
RAG
ChatGPT
Machine Learning

Instead:

```
qdrant asp.net core
ollama c#
rag asp.net core
sql server text to sql
ai database agent
semantic search c#
ollama .net
llm sql server
```

These are much closer to your actual expertise.

Google's current guidance explicitly emphasizes useful, reliable, people-first content and first-hand expertise, and warns against producing lots of automated content across many topics merely to attract search traffic.  Google for Dev... +1

So your strategy should be:

Experience → Project → Article

not:

Keyword → AI-generated article

13. Your content production system

You already have a powerful advantage.

When you're building something for yourself:

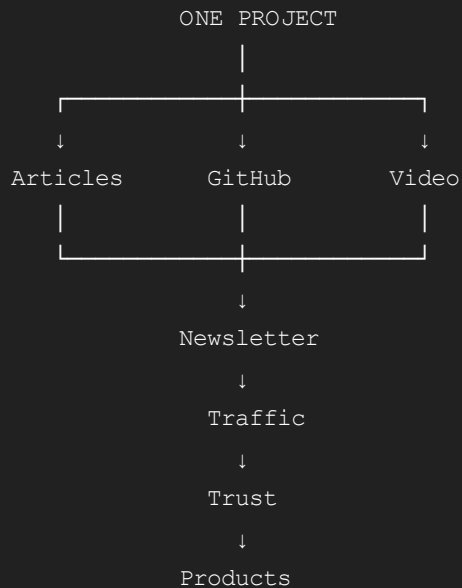
```
Real problem
  ↓
Research
  ↓
Implementation
  ↓
Problem encountered
  ↓
Solution
  ↓
```

Testing



Documentation

Turn that into:



This is the model I would follow.

14. First 90-day target

Don't obsess over traffic numbers initially.

Your first target should be:

Month 1

Website
GitHub
5 articles
1 working project
Newsletter

Month 2

10 additional articles
AI Database Agent V2
GitHub documentation
1-2 videos

Month 3

10 additional articles
RAG integration
AI Agent integration
Free downloadable resource
Newsletter consistently running

So after 90 days:

~25–30 genuinely useful articles + 1 substantial open-source project + newsletter + supporting content.

That is a much healthier foundation than trying to publish 100 AI-generated articles.

15. Budget

I'd keep your initial spending around **AED 300–500**, not AED 1,000.

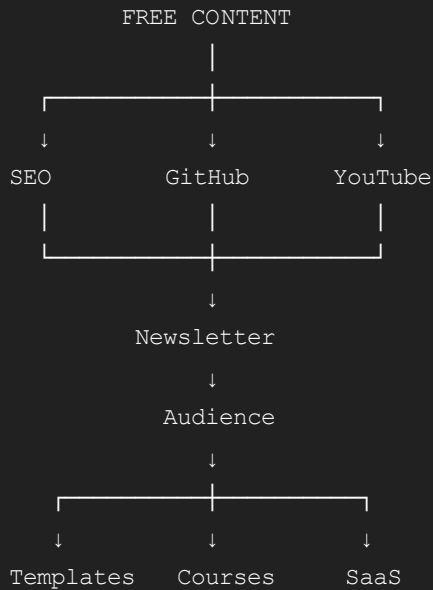
Item	Target
Domain	AED 50–100
Hosting	AED 150–250
Email	Free initially
GitHub	Free
Analytics	Free
Search Console	Free
Design	Free
AI APIs	Use local Ollama initially
Total	~AED 200–350

You already have the local AI stack, so **Ollama can dramatically reduce your early experimentation cost.**

Save the remaining budget for when you discover what actually works.

16. Your long-term business

The progression I'd aim for is:



Eventually:

Free

Articles + GitHub + videos

\$19–49

Templates / starter projects

\$49–199

Advanced project/course

\$200+

Production starter kits / consulting

SaaS

The AI Database Agent itself could eventually become a product.

17. The key strategic decision

I would **not** make the website:

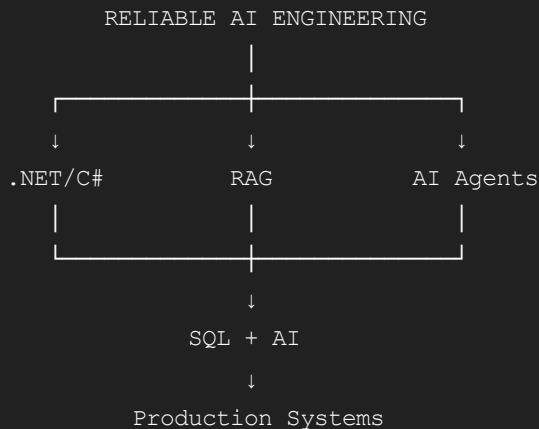
“A blog about AI.”

I'd make it:

A place where software engineers learn to build reliable AI systems.

That distinction matters.

Your five pillars become:



And your editorial philosophy:

Build → Test → Break → Fix → Explain

That is what I would build the brand around.

Phase 2

The next practical step is **not another strategy discussion**.

We should actually start building the thing.

I recommend we do it in this order:

1. Finalize brand + domain strategy
2. Choose WordPress vs custom Next.js/React site
3. Create the exact sitemap
4. Design the homepage
5. Set up GitHub
6. Design the AI Database Agent architecture
7. Build V1 of the project
8. Write Article #1 from the actual implementation
9. Set up SEO/analytics
10. Publish

For your skill level, I'd lean toward a **custom Next.js/React website rather than WordPress**, because the site itself can become another demonstration of your engineering capabilities, while your actual content remains portable. The first version

should still be deliberately simple so you don't spend three weeks building the blog instead of the AI projects.



AutoSEO

Ad

Grow organic traffic on autopilot with AutoSEO
AutoSEO AI Agents boost your business visibility using AI SEO in Google and...

Can I use google ad sense and generate revenue from there?

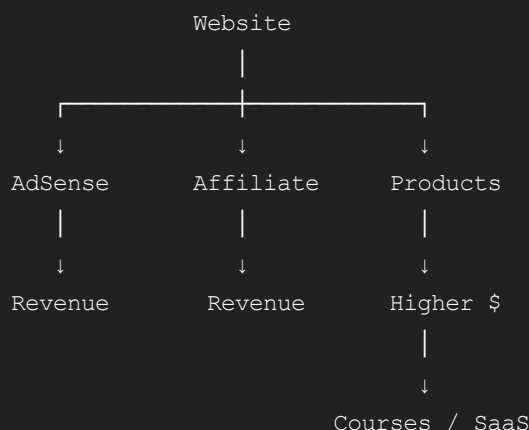
Yes — **absolutely**. AdSense can be one of the revenue streams for the website we're planning.

Google says AdSense is free to join, and publishers earn from ads shown alongside their content through clicks and/or impressions. [Google Support +1](#)

However, I would **not make AdSense the primary business model** for your AI engineering site.

How I would monetize your site

I'd use this model:



1. AdSense — passive revenue

Once you have substantial traffic, put AdSense on:

- tutorials
- guides

- architecture articles
- comparison articles
- troubleshooting articles

Google calculates RPM based on earnings per 1,000 page views/impressions, so your actual revenue depends heavily on traffic and ad performance.  Google Support +1

For example, **illustratively**, if your effective page RPM were \$5:

Monthly pageviews	Approx. monthly ad revenue
10,000	\$50
50,000	\$250
100,000	\$500
500,000	\$2,500
1,000,000	\$5,000

These are **not guaranteed AdSense rates**—just examples using a \$5 RPM assumption. Actual RPM can vary considerably.

2. Why your global audience matters

This is one reason I like your original decision to target **global developers instead of only UAE traffic**.

Your potential audience is:

USA
Canada
UK
Germany
Australia
India
UAE
Europe
Singapore
etc.

You're writing in English and targeting technical searches, so you aren't dependent on UAE traffic.

That gives you a much larger potential audience than a Dubai-only blog.

3. Don't put AdSense on the site immediately

This is important.

I'd launch like this:

Stage 1

0-20 articles

NO ADS

Focus on:

- excellent content
- SEO
- GitHub
- backlinks
- technical credibility
- Google indexing
- newsletter

Stage 2

20-50 articles

Apply for AdSense

Google says the site needs original, high-quality content and must comply with its policies. Google also says the site should be live, contain enough content for review, and be easy to navigate.  Google Support +1

There isn't a Google rule saying you need exactly "X articles" or "X visitors" before applying. So I wouldn't chase an arbitrary article count.

Instead, make the site genuinely useful first.

4. Your technical content is actually good for AdSense

Consider an article like:

"How to Build RAG with ASP.NET Core and Qdrant"

A visitor might read:

```
Introduction
  ↓
Architecture
  ↓
Qdrant setup
  ↓
ASP.NET Core implementation
  ↓
Embedding generation
  ↓
Semantic search
  ↓
Testing
  ↓
Production considerations
```

You could naturally have ads around the content without destroying the experience.

But don't overload it.

I would prefer:

excellent article + 2–4 well-positioned ads

over:

10 ads + terrible reading experience.

5. AdSense + affiliate revenue

This is where it becomes more interesting.

Your articles can recommend tools/services relevant to developers.

For example:

Article

"Build a Production RAG Application"

You could potentially recommend:

- cloud hosting
- VPS

- developer tools
- databases
- AI APIs
- monitoring tools
- developer courses
- books

Where legitimate affiliate programs are available.

So:



You're no longer dependent on advertising alone.

6. The biggest opportunity: sell your own products

This is where I think your technical background gives you a major advantage.

Imagine this article:

Build an AI Database Agent With ASP.NET Core

Free article:

Architecture
Basic implementation
Concepts
Simplified code

Then:

Premium

Production AI Database Agent Starter Kit — \$49

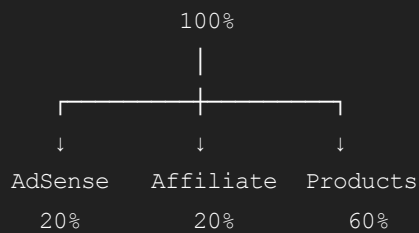
Includes:

ASP.NET Core
Clean Architecture
SQL Server
Qdrant
Ollama/OpenAI
React
Authentication
RBAC
SQL validation
Logging
Docker
Tests

Now one customer can potentially be worth more than thousands of ad impressions.

7. A realistic long-term revenue structure

I would aim for something like:



Not because those percentages are guaranteed, but because **your own products have much higher revenue potential per engaged visitor** than display advertising.

Initially it may look more like:



Then gradually:



8. How much traffic do you need?

This is the important question.

Suppose, purely as an example, your site eventually achieves a **\$5 page RPM**:

10K pageviews/month

≈ **\$50/month**

Not exciting.

50K

≈ **\$250/month**

Useful side income.

100K

≈ **\$500/month**

Now it's meaningful.

500K

≈ **\$2,500/month**

Now you're building a real media asset.

1M

≈ **\$5,000/month**

Again, these are illustrative calculations, **not forecasts**. Google defines RPM as estimated earnings divided by page views/impressions multiplied by 1,000; your actual RPM will depend on your traffic and advertising performance.  [Google Support +1](#)

9. What I would do with your AED 500–1,000

I would **not** spend the whole budget on advertising to get visitors.

Instead:

AED 300

↓

Domain + hosting

AED 0

↓

GitHub

AED 0

↓

Analytics

AED 0

↓

Google Search Console

AED 0

↓

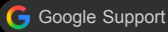
Ollama/local AI

Remaining budget

↓

Keep it for experiments

Then let **SEO + GitHub + technical content** generate your initial traffic.

Google also requires you to comply with its publisher policies and can disable ad serving or close accounts for violations. 

10. One important thing for your AI-generated content idea

Because you're an AI/software engineer, you will naturally use AI to help write articles.

That's fine as a productivity tool, but I would **not** do:

ChatGPT

↓

Generate 500 articles

↓

Publish automatically

↓

AdSense

Instead:

You build something

↓

AI helps document it

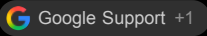
↓

You test everything

↓

You add your experience

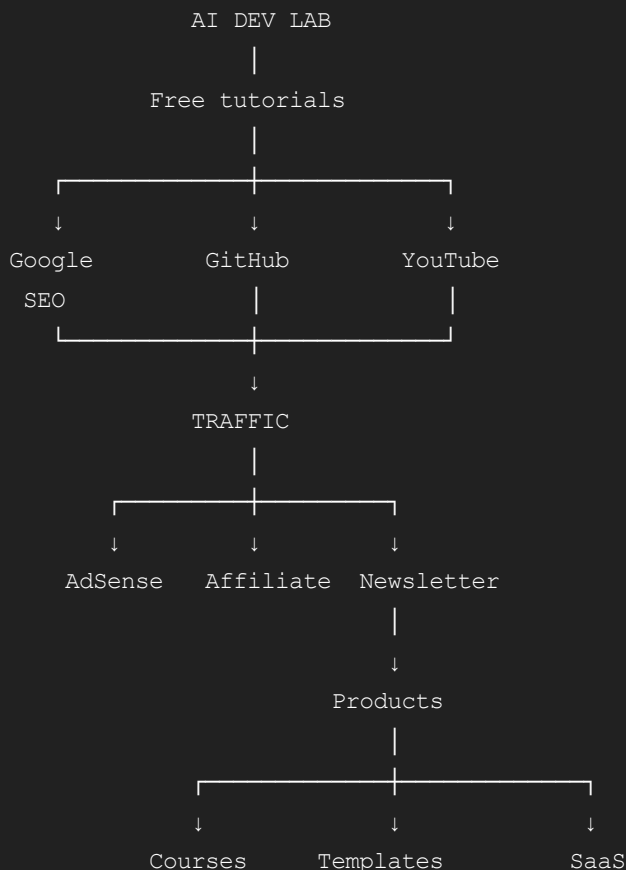
↓
You document failures
↓
Publish
↓
AdSense

That aligns much better with Google's emphasis on original, high-quality content and people-first publishing. 

My recommendation for our project

Yes, let's include AdSense from the beginning of the business plan—but not from day one of the website.

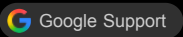
I'd build the revenue ladder as:



AdSense = traffic monetization.

Products = business monetization.

That distinction is important.

And yes, because you're based in the UAE, Google currently lists AdSense as unavailable only in certain sanctioned countries/territories; the UAE is not among those listed restrictions. 

If we proceed, I would make **AdSense** a **planned revenue channel from Month 2–3 onward**, while designing the site so it can eventually support **affiliate links, digital products, and a SaaS product** without needing a redesign.

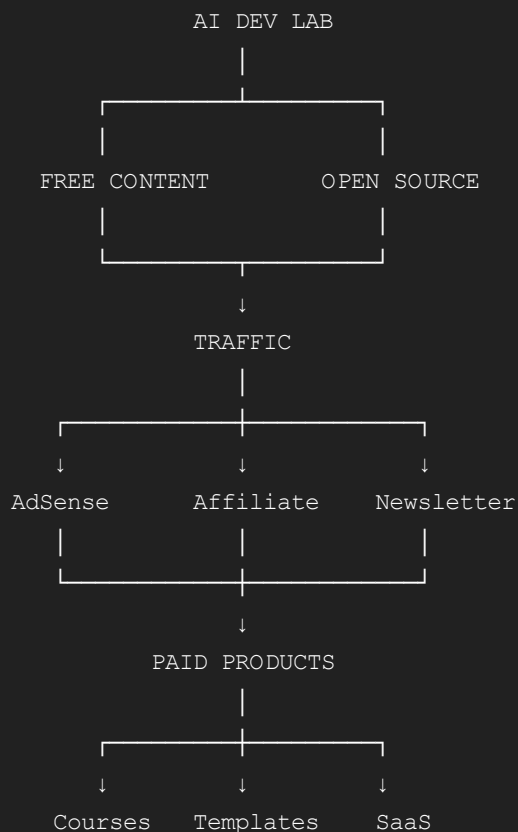
yes

Yes. Let's make **AdSense part of the architecture from day one**, while keeping it as one revenue stream rather than the entire business.

I checked Google's current documentation: AdSense is available in the **United Arab Emirates**, participation is free, and Google requires original, high-quality content and a site you control. Google reviews the entire site before ads can be shown.  Google Support +2

Our business model

I'd now define the project like this:



The important distinction

AdSense monetizes traffic.

Your products monetize expertise.

For your background, I want the second one to eventually become more important.

Phase 1 — Website

I'd build a lightweight custom site:

Recommended stack

```
Frontend
Next.js + TypeScript

Content
MDX

Styling
Tailwind CSS

Hosting
Vercel / similar

Analytics
Google Analytics

Search
Google Search Console

Monetization
Google AdSense

Code
GitHub

AI projects
.NET 8 + SQL Server + Ollama + Qdrant
```

You could also use WordPress, but since you're already a senior software engineer and React/.NET developer, the custom approach gives you more control and itself becomes a technical project.

Phase 2 — Site structure

```
/
├─ /ai-dotnet
├─ /rag
├─ /ai-agents
```

```
└─ /sql-ai
└─ /ai-tools
└─ /projects
└─ /resources
└─ /about
└─ /newsletter
```

Individual articles:

```
/rag/what-is-rag
/rag/build-rag-aspnet-core
/qdrant/qdrant-dotnet
/sql-ai/text-to-sql
/sql-ai/ai-database-agent
/ai-agents/tool-calling
/ai-dotnet/ollama-aspnet-core
```

Phase 3 — AdSense readiness

Don't launch with ads covering every paragraph.

First build:

- About page
- Contact page
- Privacy Policy
- Terms
- Cookie/consent handling where applicable
- Author information
- Clear navigation
- Original articles
- Working projects
- GitHub links
- Responsive design
- Fast pages

Google specifically says the site must be live, navigable, contain enough content for evaluation, and comply with its policies before activation.  Google Support +1

There's **no official requirement that you must have exactly 20, 30, or 50 articles** before applying. So we shouldn't manufacture articles just to hit a number.

Phase 4 — First content

Let's make the first 10 articles form a learning path.

Beginner

1. How to Build AI Applications with ASP.NET Core
2. Run Ollama Locally and Connect It to .NET
3. Build an AI Chat API with ASP.NET Core
4. What Is RAG? A Practical Guide for Developers

Intermediate

5. Build RAG with ASP.NET Core and Qdrant
6. Understanding Embeddings for .NET Developers
7. Semantic Search with Qdrant and C#

Advanced

8. Build Text-to-SQL with ASP.NET Core
9. How to Secure AI-Generated SQL
10. Build an AI Database Agent with .NET and SQL Server

The last three lead directly into your flagship project.

Phase 5 — The flagship project

We'll build this publicly:

AI Database Agent

V1

```
React
  ↓
ASP.NET Core
  ↓
Ollama
  ↓
SQL Server
```

User:

"Show me pending orders from Dubai."

LLM generates SQL.

Then:

```
SQL
↓
Validator
↓
SELECT only?
↓
Allowed tables?
↓
Row limit?
↓
Timeout?
↓
Execute
```

Then return the data.

V2

Add schema discovery:

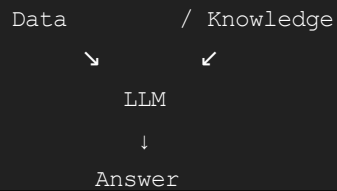
```
User
↓
AI
↓
Database Schema
↓
Relevant Tables
↓
SQL
```

This makes the project much more interesting.

V3

Add Qdrant:

```
      User
      ↓
    AI Agent
   ↙      ↘
SQL Server  Qdrant
  ↓         ↓
Structured Documents
```



V4

Add tools:

```
AI Agent
├──
├── SQL Tool
├── Vector Search Tool
├── Documentation Tool
├── Calculation Tool
```

Now you have a genuine **agentic AI project**, rather than another chatbot.

Phase 6 — Every project becomes content

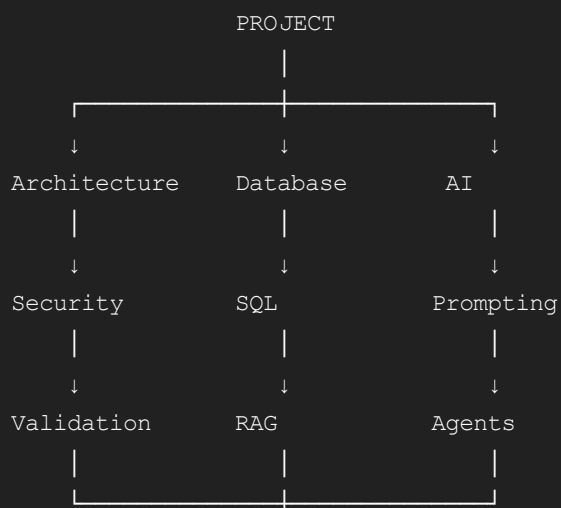
This is the key system I want you to follow.

Suppose you spend two weeks building the database agent.

Don't publish only:

"AI Database Agent Tutorial"

Instead produce:



One real project could therefore generate months of content.

Phase 7 — AdSense placement

Once approved, I'd initially use relatively conservative placements.

For a technical article:

```
| Article title  
| Introduction  
|  
  
[Ad]  
  
Architecture  
_____  
  
Code  
_____  
  
Explanation  
  
[Ad]  
  
Testing  
_____  
  
Production considerations  
  
[Ad]  
  
GitHub / Next steps
```

Don't sacrifice readability to maximize ad density.

A developer who arrives for a 20-minute tutorial and gets bombarded with ads may simply leave.

And **don't click your own ads or encourage users to do so.**

Phase 8 — AdSense isn't the only advertising opportunity

Once you have meaningful traffic, you can potentially add other monetization:

Affiliate

Example:

"Where should you host your .NET AI application?"

Potentially link to relevant hosting/cloud/developer services through legitimate affiliate programs.

Sponsored content

Later:

"Qdrant integration tutorial sponsored by X"

But keep sponsored content clearly disclosed and technically honest.

Your own products

This is where I expect the highest potential.

Product #1

AI Database Agent Starter Kit

Potential price:

\$49–\$99

Include:

```
ASP.NET Core
React
SQL Server
Qdrant
Ollama/OpenAI integration
Clean Architecture
RBAC
SQL validation
Logging
Docker
```

Product #2

Production RAG Starter Kit

Potential price:

\$49–\$149

```
.NET
React
Qdrant
SQL Server
Document ingestion
Chunking
Embeddings
Metadata
Hybrid search
Reranking
Authentication
Docker
```

Product #3

Eventually:

AI Engineering Course

Something like:

Build Production AI Applications with .NET

Modules:

```
1. LLM Fundamentals
2. Ollama
3. Embeddings
4. RAG
5. Qdrant
6. SQL + AI
7. Text-to-SQL
8. AI Agents
9. Tool Calling
10. Security
```

Phase 9 — Your revenue funnel

This is what I ultimately want:

```

Google Search
  ↓
Free article
  ↓
Useful code
  ↓
GitHub
  ↓
Newsletter
  ↓
Trust
  ↓
Free project
  ↓
Paid starter kit
  ↓
Course
  ↓
SaaS
  
```

And AdSense sits here:

```

Google Search
  ↓
Free article
  ↓
  |—— AdSense → revenue
  |
  |—— Newsletter → long-term customer
  
```

That's much stronger than:

```

Google → Article → Ad → Done
  
```

Phase 10 — 12-month target

I wouldn't promise a specific income because traffic and ad RPM are highly variable.

Instead, our milestones should be:

Month 1

Website
GitHub
5 articles

Month 2

10-15 articles
AI Database Agent V1

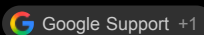
Month 3

20-25 articles
RAG implementation
Newsletter

Month 4–6

35-50 articles
AI Agent project
GitHub projects
YouTube demos
AdSense application

Google says site reviews typically take a few days, though some can take **2–4 weeks**.



Month 6–12

50-100+ high-quality articles
Multiple GitHub projects
Newsletter
AdSense
Affiliate experiments
First paid starter kit

Those article numbers are **our operating targets**, not Google requirements.

One important 2026 consideration

Google's Search documentation continues to emphasize people-first content, and its current Search guidance explicitly covers AI features and spam policies. Google's August

2026 update also reiterated enforcement against attempts to manipulate search rankings through third-party content on established sites.  Google for Dev... +1

So our strategy should be very deliberate:

❌ Don't do

AI → 1,000 generic articles → AdSense

✅ Do

```
Real engineering project
    ↓
Your implementation
    ↓
Your testing
    ↓
Your failures
    ↓
Your solutions
    ↓
Detailed article
    ↓
GitHub project
    ↓
AdSense + products
```

That is the model I'd build.

So let's actually start

I suggest we make the next step **"Phase 1A — choose and validate the brand/domain."**

I'll research **10–15 potential names**, check whether the corresponding domains appear available, check existing brands/companies that could cause confusion, and then narrow them to a small shortlist **before we spend any money**.

After that we'll build the site architecture and start the first AI Database Agent project.

For automating SEO for the site's content, here's an AutoSEO tool option.



AutoSEO

Ad

yes

I checked the names we discussed against the current web landscape. **I would change our earlier recommendation.**

Several of the obvious names are already in active use:

- **AI Engineering Lab** is already an established educational platform and has multiple other active projects using the same name.  aiengineeringla... +2
- **Build AI System** is already an active AI engineering publication/business with a very similar positioning to ours.  Build AI Sys...
- There are also existing projects using **AI Engineering Lab** as a GitHub identity.  GitHub +1

So I would not use "AI Engineering Lab" or "Build AI System."

My revised brand direction

Rather than trying to find a name that literally describes the site, I'd choose a **distinctive brand** and use the tagline to explain it.

Shortlist

Candidate	Brand idea	Tagline
BuildAI Stack	AI engineering stack	Build. Test. Ship AI.
AIForge.dev	Engineering/building	Forge reliable AI systems
AgentForge.dev	Agents + engineering	Build agents that actually work
AIBuilders.dev	Developer community	Practical AI for software engineers
DevAI Stack	Developer-focused	The practical AI engineering stack
AIWorkbench.dev	Experimental/engineering	Build, test and understand AI systems

Candidate	Brand idea	Tagline
BuildWithAI.dev	Broad	Build real software with AI
AICodeWorks.dev	Code + AI	Practical AI engineering for developers

My preference

I'd investigate:

AIForge.dev

with:

Forge reliable AI systems.

And the website positioning:

Practical AI engineering for software developers — .NET, RAG, agents, SQL, LLMs and production systems.

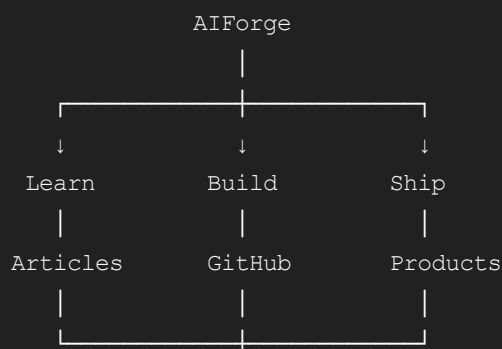
It's short, doesn't lock us to .NET or a particular AI vendor, and can support a future product company.

Important: this is a naming shortlist, not a trademark/domain clearance. We should verify the exact domain and trademark status before purchasing anything.

But there's an even better approach

I don't want us to spend your AED 500–1,000 on a domain before we've validated the concept.

Let's first build the **brand architecture**:



Then the website can eventually have:

```
aiforge.dev
|
├── Learn
|   ├── .NET AI
|   ├── RAG
|   ├── AI Agents
|   ├── SQL + AI
|   └── AI Tools
|
├── Projects
|   ├── AI Database Agent
|   ├── RAG Starter
|   └── AI Agent Starter
|
├── Resources
|
├── Newsletter
|
└── Products
```

The first product/project

I'd make this the site's flagship:

AI Database Agent

Stack:

```
React
  ↓
ASP.NET Core
  ↓
AI Orchestrator
  ↓
┌───────────┴───────────┐
│ SQL Server   Qdrant      │
│ Structured   Semantic   │
│ Data         Knowledge   │
└───────────┬───────────┘
              ↓
            Ollama
              ↓
            Answer
```

And importantly, we can build it **publicly**.

Every implementation problem becomes content.

For example, if the LLM generates:

SQL

```
SELECT *  
FROM Orders  
LIMIT 100
```

and SQL Server fails, that's an article:

Why My AI Generated `LIMIT 100` for SQL Server — and How to Prevent It

Then we explain `TOP`, schema awareness, dialect detection, validation, etc.

That is the type of content that differentiates the site from generic AI blogs.

Your first 30 articles

I'd organize them into five clusters.

.NET + AI

1. ASP.NET Core + LLMs
2. Ollama + .NET
3. Build an AI Chat API
4. Streaming LLM Responses
5. Structured LLM Responses
6. Tool Calling with .NET

RAG

7. What Is RAG?
8. Embeddings Explained
9. Chunking Strategies
10. Qdrant Fundamentals
11. Semantic Search with .NET
12. Build RAG with ASP.NET Core
13. Metadata Filtering
14. Hybrid Search
15. RAG Evaluation

SQL + AI

- 16. LLMs + SQL Server
- 17. Text-to-SQL
- 18. Schema Discovery
- 19. Validate AI-Generated SQL
- 20. Secure Text-to-SQL
- 21. AI Database Agent

AI Agents

- 22. What Is an AI Agent?
- 23. Tool Calling
- 24. Agent Memory
- 25. Agent Planning
- 26. Multi-Agent Architecture
- 27. Agent Evaluation
- 28. Agent Security

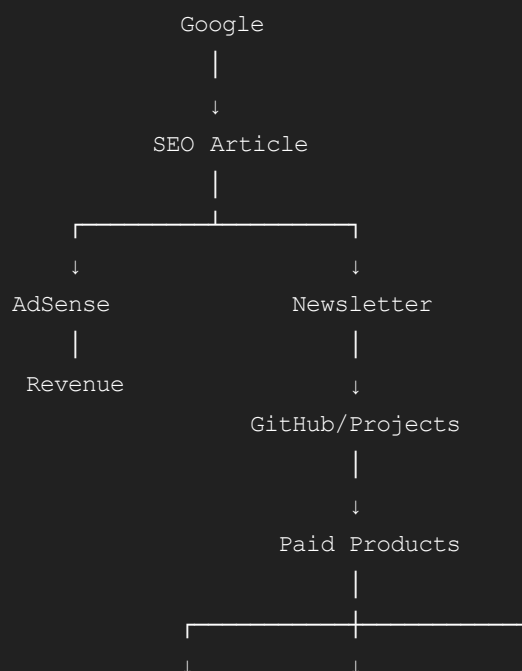
Production AI

- 29. Production RAG Architecture
- 30. Monitoring AI Applications

That gives us a **coherent SEO/content structure**, rather than 30 unrelated articles.

AdSense fits into this

The monetization architecture becomes:



Google's own guidance emphasizes original, useful content and warns against producing search-oriented content at scale without adding value. [▶ Build AI Sys...](#)

That's another reason I want the **actual engineering projects to drive the articles**.

What I recommend we do next

Rather than jumping into domain registration, let's do this in **three concrete steps**:

Step 1 — Brand validation

I'll narrow the shortlist to **3 names** and investigate:

- domain availability
- existing websites
- GitHub usage
- obvious company conflicts
- social handle conflicts

Step 2 — Choose the winner

Then define:

- logo direction
- tagline
- color direction
- site positioning
- About statement

Step 3 — Build the website

Then we'll create:

```
Homepage
  ↓
Navigation
  ↓
Article system
  ↓
Projects
  ↓
Newsletter
  ↓
AdSense-ready structure
```

After that, we start building the **AI Database Agent V1** and use the actual project to produce Article #1.

I would start with `AIForge.dev`, `BuildAI Stack`, and `AIWorkbench.dev` as the three candidates to investigate further.

For building traffic, an AI-assisted SEO platform is another option.


Google & AI Search
On autopilot.


 SEOPro AI
SEO for Google & AI
Grow organic traffic from Gemini & ChatGPT

Ad

ChatGPT can make mistakes. Check important info.